

Figure 1: Xenomai project architecture

Warning: The driver is still under testing so if any bug is found, please inform me.

The I2C driver is a platform driver and hence some setting is required in the device tree of the Cortex-A8 processor which resides in Beagle-Bone Black. Various real-time driver development APIs are available like, device registration services, task services, clock services, timer services, synchronisation services, interrupt management, etc.

Registering the RTDM device API

Before registering the RTDM driver, we need to define its structure `struct rtdm_device`. The following is struct `rtdm_device` initialisation for the RTDM driver for I2C:

```
static struct rtdm_device i2c_device= {
    .struct_version= RTDM_DEVICE_STRUCT_VER,
    .device_flags= RTDM_NAMED_DEVICE,
    .context_size= sizeof(MY_DEV), //size of local structure variable
    .device_name= "omap-i2c",
    .proc_name= "omap-i2c",
    .open_nrt= i2c_open_nrt,
    .ops={
        .close_nrt = i2c_close_nrt,
        .read_rt= i2c_rd_rt,
        .write_rt= i2c_wr_rt,
        .ioctl_rt=i2c_ioctl_rt,
    },
    .device_class=RTDM_CLASS_SERIAL,
    .device_sub_class=2015,
    .profile_version=1,
    .driver_name=DRIVER_NAME,
    .driver_version=RTDM_DRIVER_VER(0,0,1),
    .peripheral_name="RTDM I2C MASTER",
    .provider_name="JAY KOTHARI",
};
```

Now we need to register this information using API `int rtdm_dev_register (struct rtdm_device * device)`.

Example:

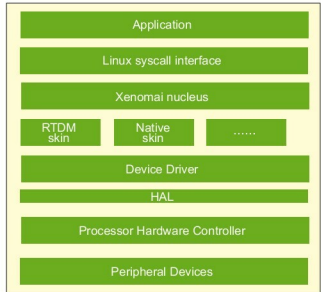


Figure 2: Real time driver model architecture

```
struct rtdm_device *rdev;
rtdm_dev_register(rdev);
```

Interrupt management API

Now we need to register the interrupt number which we got from the `platform_get_irq()` API, as follows:

```
int rtdm_irq_request ( rtdm_irq_t * irq_handle,
    unsigned int irq_no,
    rtdm_irq_handler_t handler,
    unsigned long flags,
    const char * device_name,
    void * arg )
```

Example:

```
err=rtdm_irq_request (&dev->irq_handle,dev->irq,rtdm_my_isr_2,0,pdev->name,dev);
```

With the above API, when the hardware interrupt occurs, the Xenomai co-kernel checks the interrupt table and calls the ISR which is registered with that interrupt.

Synchronisation management API

A locking mechanism can be initialised with the following API:

```
rtdm_lock_init(rtdm_lock_t lock )
```

Example:

```
rtdm_lock_t lock;
rtdm_lock_init(lock);
rtdm_lock_irqrestore(rtdm_lockctx_t context ) : Restores
```